Memoria práctica 2

Seguridad de la información

Luis Martín Pérez – curso 2025/2026

Índice

1 Generación y comprobación de resúmenes....3
1.1 Crear un archivo de texto legible entre 150 y 200 caracteres....3
1.2 Aplicar un mínimo de cinco algoritmos de resumen, verificar que el tamaño coincide con el algoritmo y comprobar como varían los resultados ante modificaciones mínimas....3
1.3 Alternar entre la salida hexadecimal, hexadecimal con : y binaria....7
2 Cifrado simétrico de documentos....8
2.1 Crear un archivo de texto legible de tamaño impar, entre 31 y 81 caracteres....8
2.2 Cifrarlo con siete algoritmos simétricos....8
2.3 Descifrarlos y comprobar el resultado....9
2.4 Explicar el tamaño de los ficheros cifrados en virtud del tamaño de bloque del cifrador, y sabiendo que la sal añade 16 bytes extra al principio del fichero....11
2.5 Cifrar el archivo con AES-256-CBC y contraseña y descifrarlo con el conjunto clave y vector de inicialización....12
3 Aplicación: Demostración de la peligrosidad del modo de operación ECB....13
3.1 Escoger una imagen de entre 20 y 50 KB de colores sólidos....13
3.2 Con la utilidad convert del paquete *ImageMagick* la convertiremos a formato PGM....13
3.3 Separaremos la cabecera de 3 líneas del cuerpo binario....14
3.4 Cifraremos el cuerpo con el cifrador de bloques AES-256 en modo ECB, si se usa contraseña, realizar el cifrado sin sal....14
3.5 Crearemos la nueva imagen con la cabecera anterior y el cuerpo cifrado....15
3.6 Convertiremos la nueva imagen al formato original....15
3.7 Verificaremos que la imagen es visible, como en el ejemplo del pingüino Tux....15
3.8 Realizar la operación con otro cifrado de 64 bits de bloque en modo ECB y comparar los resultados....15
Bibliografía....17

1 Generación y comprobación de resúmenes

1.1 Crear un archivo de texto legible entre 150 y 200 caracteres

Con la utilidad `echo` se escribe un texto cualquiera en el fichero `plain-text.txt`.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2]
└─$ echo "Sing, O goddess, the anger of Achilles son of Peleus, that brought
countless ills upon the Achaeans, and sent many valiant souls of heroes down
to Hades" > plain-text.txt
```

Figura 1.1a – Creación del fichero de texto plano a partir de los primeros caracteres de "La Ilíada".

1.2 Aplicar un mínimo de cinco algoritmos de resumen, verificar que el tamaño coincide con el algoritmo y comprobar como varían los resultados ante modificaciones mínimas

Los algoritmos de resúmenes escogidos son los siguientes:

- `md5`
- `sha1`
- `sha256`
- `sha3-512`
- `whirlpool`

El comando para generar resúmenes en openssl es bastante simple

```
$ openssl dgst -{nombre del algoritmo} -{binary/hex} fichero-texto-plano
```

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2]
└─$ openssl dgst -md5 -binary plain-text.txt > text.md5

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2]
└─$ openssl dgst -sha1 -binary plain-text.txt > text.sha1

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2]
└─$ openssl dgst -sha256 -binary plain-text.txt > text.sha256

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2]
└─$ openssl dgst -sha3-512 -binary plain-text.txt > text.sha3-512

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2]
└─$ openssl dgst -whirlpool -binary plain-text.txt > text.whirlpool
```

Figura 1.2.a – Aplicando los cinco algoritmos en binario.

A continuación, se verifican los tamaños de los ficheros en bytes.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2]
└─$ ls -lh
total 48K
-rw-r--r-- 1 kali kali 22K Feb 12 17:46 burger-king.png
-rw-rw-r-- 1 kali kali 153 Feb 12 19:16 plain-text.txt
-rw-rw-r-- 1 kali kali  16 Feb 12 19:31 text.md5
-rw-rw-r-- 1 kali kali  20 Feb 12 19:31 text.sha1
-rw-rw-r-- 1 kali kali  32 Feb 12 19:32 text.sha256
-rw-rw-r-- 1 kali kali  64 Feb 12 19:33 text.sha3-512
-rw-rw-r-- 1 kali kali  64 Feb 12 19:33 text.whirlpool
```

Figura 1.2.b – Lista de cada resumen y su tamaño en bytes.

Según la información recogida en la página de Wikipedia[1] sobre funciones de resumen, se obtiene la siguiente tabla:

Algoritmo	Tamaño resumen (bits)	Tamaño resumen (Bytes)
MD5	128	16
SHA-1	160	20
SHA-256	256	32
SHA3-512	512	64
Whirlpool	512	64

Como es de esperar, cada fichero resultante tiene el tamaño que le corresponde. A continuación, se genera otro fichero de texto plano a partir de la modificación del original.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2]
└─$ diff plain-text2.txt plain-text.txt
1c1
< Sing, i goddess, the anger of Achilles son of Peleus, that brought countless ills upon the
---
> Sing, O goddess, the anger of Achilles son of Peleus, that brought countless ills upon the
```

Figura 1.2.c – Comparación de ficheros de texto plano.

Se cambia solo un byte, es decir, la letra "*O*" por la letra "*i*" y se comprueba que el resumen con MD5 es totalmente diferente en la siguiente figura.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2]
└─$ openssl dgst -md5 -binary plain-text2.txt > text2.md5

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2]
└─$ diff <(xxd text.md5) <(xxd text2.md5)
1c1
< 00000000: 244d a6d8 2abe c0fa cb4c 7f35 4dd7 ea0d  $M..*....L.5M...
---
> 00000000: bf5d 616d 6f04 a335 7cf5 95b7 4b5e da80  .]amo..5|...K^..
```

Figura 1.2.d – Comprobación del efecto avalancha al cambiar un byte en MD5.

La única coincidencia es medio byte, lo cuál lo convierte en otro resumen completamente diferente. Si se hace el mismo experimento, pero en lugar de cambiar un byte entero, se cambia un sólo bit.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2]
└─$ diff <(xxd plain-text.txt) <(xxd plain-text2.txt)
1c1
< 00000000: 5369 6e67 2c20 4f20 676f 6464 6573 732c  Sing, O goddess,
---
> 00000000: 5369 6e67 2c20 4e20 676f 6464 6573 732c  Sing, N goddess,
```

Figura 1.2.e – Comparación de ficheros de texto plano al cambiar un bit.

El código ASCII del carácter “O” es `0x4f`, si se le resta uno se obtiene `0x4e`, el carácter “N”. Al generar el resumen con los cuatro algoritmos restantes se puede comprobar que ningún resumen es coincide en más de un byte seguido. Con esto se demuestra que estos algoritmos tienen una capacidad de dispersión muy buena.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2]
└─$ openssl dgst -sha1 -binary plain-text2.txt > text2.sha1

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2]
└─$ diff <(xxd text.sha1) <(xxd text2.sha1)
1,2c1,2
< 00000000: 2508 6fa8 ddbb bd9f 8d0c 4037 e1e3 bf56  %.o.......@7...V
< 00000010: 7a30 f8c3                                z0..
---
> 00000000: e189 0efd c9fb 745b 2a16 ed8e f202 8d5a  ......t[*......Z
> 00000010: 5415 1e26                                T..&
```

Figura 1.2.f – Comparación de resúmenes con SHA-1 al cambiar un bit.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2]
└─$ openssl dgst -sha256 -binary plain-text2.txt > text2.sha256

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2]
└─$ diff <(xxd text.sha256) <(xxd text2.sha256)
1,2c1,2
< 00000000: b5eb afb2 14d8 4fed 763d 083b c13a a69e  ......O.v=.;.:..
< 00000010: 5f13 1527 fe0a e74d c37c e3c3 29e5 ddbb  _..'...M.|..)...
---
> 00000000: 4e8e 75b0 1aeb 1ee9 5eb2 816d 0180 b871  N.u.....^..m...q
> 00000010: 7469 1482 1182 7e42 b88e 8fc4 f554 261a  ti....~B.....T&.
```

Figura 1.2.g – Comparación de resumenes con SHA-256 al cambiar un bit.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2]
└─$ openssl dgst -sha3-512 -binary plain-text2.txt > text2.sha3-512

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2]
└─$ diff <(xxd text.sha3-512) <(xxd text2.sha3-512)
1,4c1,4
< 00000000: ff3d 7640 dff5 ead2 1975 44dc d659 c862  .=v@.....uD..Y.b
< 00000010: f8ef 49a3 54c4 e814 6532 85a6 7cf9 b7db  ..I.T...e2..|...
< 00000020: 6988 4968 ea9c 93f4 bf19 4650 8ce2 90e3  i.Ih......FP....
< 00000030: f04a 4467 e7fd d5c7 64e1 b600 88b7 ca2f  .JDg....d...../
---
> 00000000: a04a 6e9c eca7 0cee 0c4e c93c 0af1 1a95  .Jn......N.<....
> 00000010: 19be d365 edc7 721c b972 12a5 e518 9cd0  ...e..r..r......
> 00000020: b73f 1691 4ee9 6288 72da 5c8f c390 e8b6  .?..N.b.r.\.....
> 00000030: 2784 1e56 6e3c d23a 6f45 cf6a 802c 5de1  '..Vn<.:oE.j.,].
```

Figura 1.2.h – Comparación de resumenes con SHA-3-512 al cambiar un bit.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2]
└─$ openssl dgst -whirlpool -binary plain-text2.txt > text2.whirlpool

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2]
└─$ diff <(xxd text.whirlpool) <(xxd text2.whirlpool)
1,4c1,4
< 00000000: dee1 d4a9 29f3 10d0 f9e6 607e ffce 3b36  ....).....`~..;6
< 00000010: aa36 65d3 55ae 150c 2ece 72f3 1523 29fd  .6e.U.....r..#).
< 00000020: 0420 c365 b928 5ee0 c220 e147 a63f 290d  . .e.(^.. .G.?).
< 00000030: dd64 d807 a5e6 146e fdb3 398a c316 2fdc  .d.....n..9.../.
---
> 00000000: c895 6ced 2d9d ed15 81cc 513e de0f 0b21  ..l.-.....Q>...!
> 00000010: eb88 0efa 3b9d eb26 84a9 406b 385c ff24  ....;..&..@k8\.$
> 00000020: f0eb cdfa 9b2f 048b 14ff 57e4 080a 8ad9  ...../....W.....
> 00000030: 655b f34d 0d30 f01f abea 1228 bbcb c3c9  e[.M.0.....(....
```

Figura 1.2.i – Comparación de resumenes con Whirlpool al cambiar un bit.

1.3 Alternar entre la salida hexadecimal, hexadecimal con : y binaria

Para obtener una salida hexadecimal, se añade la opción `-hex`, en vez de `-binary` y para añadir dos puntos entre cada byte, se añade `-c`. La opción de hexadecimal genera como resultado un fichero binario, sino la representación del resumen en caracteres que representan en hexadecimal el resumen binario. Se demuestra con las siguientes figuras.

```
$ openssl dgst -md5 -hex plain-text.txt > text.hex.md5
```

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2]
└─$ cat text.hex.md5
MD5(plain-text.txt)= 244da6d82abec0facb4c7f354dd7ea0d

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2]
└─$ xxd text.md5
00000000: 244d a6d8 2abe c0fa cb4c 7f35 4dd7 ea0d  $M..*....L.5M..
```

Figura 1.3.a – Demostración de que la opción hex genera texto plano.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2]
└─$ cat text.hex.c.whirlpool | fold -w 64
WHIRLPOOL(plain-text.txt)= de:e1:d4:a9:29:f3:10:d0:f9:e6:60:7e:f
f:ce:3b:36:aa:36:65:d3:55:ae:15:0c:2e:ce:72:f3:15:23:29:fd:04:20
:c3:65:b9:28:5e:e0:c2:20:e1:47:a6:3f:29:0d:dd:64:d8:07:a5:e6:14:
6e:fd:b3:39:8a:c3:16:2f:dc

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2]
└─$ xxd text.whirlpool
00000000: dee1 d4a9 29f3 10d0 f9e6 607e ffce 3b36  ....).....`~..;6
00000010: aa36 65d3 55ae 150c 2ece 72f3 1523 29fd  .6e.U....r..#).
00000020: 0420 c365 b928 5ee0 c220 e147 a63f 290d  . .e.(^.. .G.?).
00000030: dd64 d807 a5e6 146e fdb3 398a c316 2fdc  .d.....n..9.../.
```

Figura 1.3.b – Visualización del resumen en hexadecimal con dos puntos de separación.

2 Cifrado simétrico de documentos

2.1 Crear un archivo de texto legible de tamaño impar, entre 31 y 81 caracteres.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2/ej2]
└─$ echo "The sun sets slowly over the quiet hill." > message

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2/ej2]
└─$ ls -lh
total 4.0K
-rw-rw-r-- 1 kali kali 41 Feb 25 16:04 message
```

Figura 2.1.a – Creación de un mensaje de texto plano de tamaño impar.

2.2 Cifrarlo con siete algoritmos simétricos.

Los algoritmos elegidos son los siguientes.

Algoritmo	Modo	Tipo
AES-256	CBC	Bloque
AES-256	CFB	Flujo
TDES	ECB	Bloque
TDES	OFB	Flujo
ChaCha20	-	Flujo
RC4	-	Flujo
Camellia-256	CFB	Flujo

En las siguientes figuras se muestran las secuencias de cifrado asimétrico.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2/ej2]
└─$ openssl enc -aes-256-cfb -in message -out message.aes-256-cfb -pbkdf2
enter AES-256-CFB encryption password:
Verifying - enter AES-256-CFB encryption password:

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2/ej2]
└─$ openssl enc -aes-256-cbc -in message -out message.aes-256-cbc -pbkdf2
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
```

Figura 2.2.a – Cifrado con AES en modo CBC y CFB.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2/ej2]
└─$ openssl enc -des-ede3 -in message -out message.tdes -pbkdf2
enter DES-EDE3-ECB encryption password:
Verifying - enter DES-EDE3-ECB encryption password:

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2/ej2]
└─$ openssl enc -des-ede3-ofb -in message -out message.tdes-ofb -pbkdf2
enter DES-EDE3-OFB encryption password:
Verifying - enter DES-EDE3-OFB encryption password:
```

Figura 2.2.b – Cifrado con TDES en modo ECB y OFB.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2/ej2]
└─$ openssl enc -chacha20 -in message -out message.chacha20 -pbkdf2
enter ChaCha20 encryption password:
Verifying - enter ChaCha20 encryption password:

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2/ej2]
└─$ openssl enc -rc4 -in message -out message.rc4 -pbkdf2
enter RC4 encryption password:
Verifying - enter RC4 encryption password:
```

Figura 2.2.c – Cifrado con los algoritmos de flujo ChaCha20 y RC4.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2/ej2]
└─$ openssl enc -camellia-256-cfb -in message -out message.camellia-cfb -pbkdf2
enter CAMELLIA-256-CFB encryption password:
Verifying - enter CAMELLIA-256-CFB encryption password:
```

Figura 2.2.d – Cifrado con Camellia en modo CFB.

Se eligió Camellia como séptimo cifrado debido a que, según Wikipedia[2], contiene propiedades criptográficas similares a las de AES y es usado por el gobierno japonés y empresas japonesas como *Mitsubishi* o *NTT*. El cifrado ha sido aprobado por ISO y NESSIE.

2.3 Descifrarlos y comprobar el resultado.

En el proceso de descifrado, el comando es prácticamente el mismo, lo que cambia es la adición de la opción `-d`.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2/ej2]
└─$ openssl enc -aes-256-cbc -d -in message.aes-256-cbc -pbkdf2
enter AES-256-CBC decryption password:
The sun sets slowly over the quiet hill.

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2/ej2]
└─$ openssl enc -aes-256-cfb -d -in message.aes-256-cfb -pbkdf2
enter AES-256-CFB decryption password:
The sun sets slowly over the quiet hill.

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2/ej2]
└─$ openssl enc -des-ede3-ofb -d -in message.tdes-ofb -pbkdf2
enter DES-EDE3-OFB decryption password:
The sun sets slowly over the quiet hill.

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2/ej2]
└─$ openssl enc -des-ede3-ecb -d -in message.tdes -pbkdf2
enter DES-EDE3-ECB decryption password:
The sun sets slowly over the quiet hill.

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2/ej2]
└─$ openssl enc -rc4 -d -in message.rc4 -pbkdf2
enter RC4 decryption password:
The sun sets slowly over the quiet hill.

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2/ej2]
└─$ openssl enc -chacha20 -d -in message.chacha20 -pbkdf2
enter ChaCha20 decryption password:
The sun sets slowly over the quiet hill.
```

Figura 2.3.a – Descifrado de los seis primeros algoritmos.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2/ej2]
└─$ openssl enc -camellia-256-cfb -d -in message.camellia-cfb -pbkdf2
enter CAMELLIA-256-CFB decryption password:
The sun sets slowly over the quiet hill.
```

Figura 2.3.b – Descifrado del mensaje cifrado con Camellia.

2.4 Explicar el tamaño de los ficheros cifrados en virtud del tamaño de bloque del cifrador, y sabiendo que la sal añade 16 bytes extra al principio del fichero.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2/ej2]
└─$ ls -lh
total 32K
-rw-rw-r-- 1 kali kali 41 Feb 25 16:04 message
-rw-rw-r-- 1 kali kali 64 Feb 25 16:13 message.aes-256-cbc
-rw-rw-r-- 1 kali kali 57 Feb 25 16:11 message.aes-256-cfb
-rw-rw-r-- 1 kali kali 57 Feb 25 16:27 message.camellia-cfb
-rw-rw-r-- 1 kali kali 57 Feb 25 16:20 message.chacha20
-rw-rw-r-- 1 kali kali 57 Feb 25 16:20 message.rc4
-rw-rw-r-- 1 kali kali 64 Feb 25 16:16 message.tdes
-rw-rw-r-- 1 kali kali 57 Feb 25 16:17 message.tdes-ofb
```

Figura 2.4.a – Lista de ficheros cifrados y su correspondiente tamaño.

En la siguiente tabla se muestra el tamaño de bloque de cada algoritmo de bloque.

Algoritmo	Tamaño de bloque	Tamaño del resultado	Tamaño sin 16 bytes	Número de bloques del fichero original
AES-CBC	128 bits, 16 bytes	64 bytes	48 bytes	41 / 16 = 2.51
TDES-ECB	64 bits, 8 bytes	64 bytes	48 bytes	41 / 8 = 5.125

Los cifrados de bloque después de cifrar los 41 bytes del fichero original, añaden 7 bytes de más para completar el último bloque, en el caso de AES, el mensaje ocupa 3 bloques y, 6 bloques con TDES. Como 48 es múltiplo del tamaño de bloque de ambos cifrados (16 y 8), ambos redondean hacia 48. Si el tamaño del mensaje original fuese de 49 bytes, el tamaño del resultado con el cifrado TDES, el siguiente múltiplo de 8 sería 56, y con el cifrado AES, el siguiente múltiplo de 16 sería 64, por lo tanto sus tamaños de mensajes cifrados serían 72 y 80 respectivamente al añadir los bytes extra (véase figura 2.4.b). Respecto a los cifrados de flujo, como la clave se deriva tan larga como el tamaño del mensaje, el tamaño del mensaje cifrado es el mismo si se le restan los 16 primeros bytes, 57 – 16 = 41.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2/ej2]
└─$ diff message message2
1c1
< The sun sets slowly over the quiet hill.
---
> The sun sets slowly over the quiet hill at dusk.

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2/ej2]
└─$ ls -lh message2*
-rw-rw-r-- 1 kali kali 49 Feb 27 15:13 message2
-rw-rw-r-- 1 kali kali 80 Feb 27 15:18 message2.aes-256-cbc
-rw-rw-r-- 1 kali kali 72 Feb 27 15:18 message2.tdes
```

Figura 2.4.b – Verificación de tamaño de bloque de AES y TDES

2.5 Cifrar el archivo con AES-256-CBC y contraseña y descifrarlo con el conjunto clave y vector de inicialización.

Si se cifra con el mismo comando de antes, pero añadiendo la opción `-p`, openssl muestra por `stdout` el conjunto de la sal, la clave y el iv.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2/ej2]
└─$ openssl enc -aes-256-cbc -p -in message -out message.aes-256-cbc -pbkdf2
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
salt=1C1A2C243D57133B
key=D337CD8CCC632956D356E885D1E2DF866A3C88C338DE02AC3A886F3D9BDF4550
iv =755F1107AD07FE62A4B1E55083CD2F05

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2/ej2]
└─$ dd bs=1 skip=16 if=message.aes-256-cbc of=message-no-salt.aes-256-cbc
48+0 records in
48+0 records out
48 bytes copied, 8.8204e-05 s, 544 kB/s
```

Figura 2.5.a – Mensaje cifrado, pero mostrando la clave y el vector generados.

A continuación, si se descifra añadiendo las opciones `-K` y `-iv`, donde se incluye la clave y el vector respectivamente, se descifra el mensaje sin necesidad de la contraseña.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2/ej2]
└─$ openssl enc -d -aes-256-cbc -K D337CD8CCC632956D356E885D1E2DF866A3C88C338DE02AC3A886F3D9BDF4550 -iv 755F1107AD07FE62A4B1E
55083CD2F05 -in message-no-salt.aes-256-cbc
The sun sets slowly over the quiet hill.
```

Figura 2.5.b – Mensaje descifrado a partir de la combinación clave y vector.

3 Aplicación: Demostración de la peligrosidad del modo de operación ECB

3.1 Escoger una imagen de entre 20 y 50 KB de colores sólidos

La imagen en la figura 3.1.a fue seleccionada de una página que contiene iconos[3], debido a que suelen ser imágenes vectoriales y, por lo tanto, de colores sólidos.

Figura 3.1.a – Icono de ejemplo para la demostración.

Se descargó la imagen en tamaño 480x480px para cumplir con el tamaño mínimo, como se puede ver en la figura 3.1.b.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2/ej3]
└─$ ls -lh
total 24K
-rw-r--r-- 1 kali kali 22K Feb 12 17:46 burger-king.png
```

Figura 3.1.b – Imagen de 22KB listada en el sistema de ficheros.

3.2 Con la utilidad `convert` del paquete *ImageMagick* la convertiremos a formato PGM

Tras verificar que el paquete ImageMagick se encuentra instalado, se crea una nueva imagen en format PGM a partir de la original.

```
$ man ImageMagick # verificar que el paquete existe y ver las utilidades
$ convert burger-king.png burger-king.pgm
```

La imagen generada se puede observar en la figura 3.2.a.

Figura 3.2.a – Imagen de prueba en el format PGM (Portable GreyMap).

3.3 Separaremos la cabecera de 3 líneas del cuerpo binario

Como se muestra en la figura 3.3.a, al mostrar el contenido aparecen 3 líneas antes de la información binaria. En la primera línea aparece P5, que indica que se trata de una imagen en escala de grises y los datos de la imagen están en binario (número mágico P5), la segunda línea representa el tamaño (480x480px) y la escala está representada como el rango de 0 a 255 en la tercera línea, 0 para el negro y 255 para el blanco[4].La cuarta línea contiene a modo de mapa de bits la información de la imagen en binario.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-2/ej3]
└─$ cat burger-king.pgm
P5
480 480
255
```

Figura 3.3.a – Cabecera de la imagen de prueba en formato PGM.

Para separar el cuerpo binario se abre la imagen con `vim` y se teclea el siguiente comando de `vi`.

```
:4w pgm-body | 4d
```

Comando	Acción que representa
`:4w pgm-body`	Escribir la línea 4 en el fichero `pgm-body`
`4d`	Borrar la línea 4

3.4 Cifraremos el cuerpo con el cifrador de bloques AES-256 en modo ECB, si se usa contraseña, realizar el cifrado sin sal

Se cifra de la misma manera que en el ejercicio 2, esta vez se vuelca el resultado como nuevo cuerpo de la imagen en formato PGM.

```
$ openssl enc -aes-256-ecb -nosalt -in pgm-body -pbkdf2 >> burger-king.pgm
```

3.5 Crearemos la nueva imagen con la cabecera anterior y el cuerpo cifrado

Este paso se resuelve junto al paso anterior, como anteriormente eliminamos el cuerpo original de la imagen, se añade la versión cifrada directamente en la imagen PGM.

3.6 Convertiremos la nueva imagen al formato original

Esta vez se realiza la conversión al revés, de PGM a PNG.

```
$ convert burger-king.pgm buger-king-ecb.png
```

3.7 Verificaremos que la imagen es visible, como en el ejemplo del pingüino Tux.

En la figura 3.7.a se puede ver como el logo es bastante distinguible, y si se mira con los ojos entre abiertos, todavía se pueden leer los caracteres.

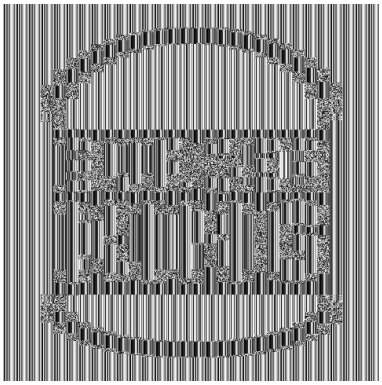

Figura 3.7.a – Visualización del logo cifrado con AES-256-ECB

3.8 Realizar la operación con otro cifrado de 64 bits de bloque en modo ECB y comparar los resultados.

Se repiten los pasos de los apartados 3.2 y 3.3, y se cifra el cuerpo de la siguiente manera con DES.

```
$ openssl enc -des-ecb -in pgm-body -nosalt -pbkdf2 >> pgm-body.des
```

Se vuelven a repetir los pasos de los apartados 3.5 y 3.6 para obtener la imagen de la figura 3.8.a.

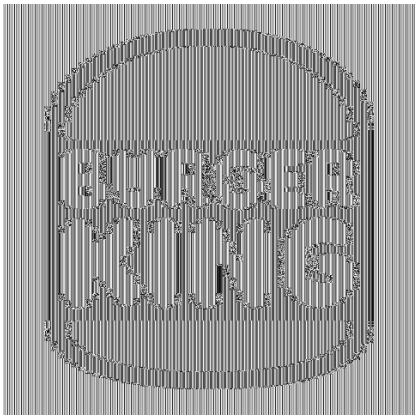

Figura 3.8.a – Visualización del logo cifrado con DES-ECB

Se comprueban la especulaciones, es decir, un menor tamaño de bloque hace que haya más contraste entre cambios de color o de forma, lo que resulta en una imagen cifrada drásticamente más visible. Si se repitiese el experimento con una imagen de una resolución más grande, el resultado se vería más claro.

Bibliografía

[1] *Wikipedia contributors, "Cryptographic hash function," Wikipedia, The Free Encyclopedia, [Online]. Available: https://en.wikipedia.org/wiki/Cryptographic_hash_function#Cryptographic_hash_algorithms. [Accessed: Feb. 25, 2026].*

[2] *Wikipedia contributors, "Camellia (cipher)," Wikipedia, The Free Encyclopedia, [Online]. Available: https://en.wikipedia.org/wiki/Camellia_(cipher). [Accessed: Feb. 25, 2026].*

[3] *Icons8, "Free Icons - Download 1,491,800 Free Icons (SVG, PNG)," Icons8, 2026. [Online]. Available: https://icons8.com/icons. [Accessed: Feb. 26, 2026].*

[4] *Wikipedia contributors, "Netpbm," Wikipedia, The Free Encyclopedia, [Online]. Available: https://en.wikipedia.org/wiki/Netpbm#Description. [Accessed: Feb. 26, 2026].*